

打刻記録が編集申請画面に反映されない問題の修正

この問題の原因は、「打刻の保存先（ AttendanceRecord ）」と「編集カレンダーの読み込み先（ EntryExitCalendar ）」という2つの帳簿の間でパイプラインがまだ繋がっていないことにあります。

この問題を解消するためのトラフィック（データの流れ）の改善設計と、具体的な修正コードをご紹介します。

データのトラフィック（修正方針）の考察

打刻を行った瞬間に、裏側で以下のような連動（トラフィックの拡張）を発生させるのがベストです。

[フロントで打刻ボタン押下]



1. AttendanceController.punch() が呼ばれる



2. AttendanceRecordService.savePunch() でログ形式の打刻を保存（既存処理）



★【ここを新しく追加！】

3. 同じ Service 内で EntryExitCalendarRepository を呼び出し、
当日のカレンダー行（EntryExitCalendar）に打刻時刻を「確定時刻」として自動で同期・上書き保存

このように、「打刻ログが保存されたら、同時にカレンダー側（当日分）の確定Entry/Exit時刻も一緒に更新する」という裏連動を1箇所加えるだけで、フロントエンドの画面を切り替えた際にも完全に一致した最新データが表示されるようになります。

バックエンド（Java）側の修正コード

既存の `AttendanceRecordService.java` を修正して、カレンダーテーブル（`entry_exit_calendar`）へ打刻をリアルタイムに流し込む記述を追加しましょう。

`AttendanceRecordService.java` の修正・差し替え

クラスの最初の方に `EntryExitCalendarRepository` を呼び出すための `@Autowired` を1行追加し、`savePunch` メソッドの末尾に同期ロジックを付け足します。

```
package com.appspace.backend.service;

import com.appspace.backend.entity.AttendanceRecord;
import com.appspace.backend.entity.EntryExitCalendar; // ★追加
import com.appspace.backend.repository.AttendanceRecordRepository;
import com.appspace.backend.repository.EntryExitCalendarRepository; // ★追加
import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Autowired; // ★追加
import org.springframework.stereotype.Service;
```

```
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDateTime;
import java.time.LocalDate; // ★追加
import java.time.format.DateTimeFormatter; // ★追加
import java.util.List;
import java.util.Optional;
import java.util.UUID; // ★追加

@Service
@RequiredArgsConstructor
@Transactional
public class AttendanceRecordService {

    private final AttendanceRecordRepository repository;

    // 翻訳窓口（カレンダー用リポジトリ）をここに接続します
    @Autowired
    private EntryExitCalendarRepository calendarRepository;

    /* --- 中略（clockIn, clockOut, getHistoryByAccountId 等はそのまま） --- */

    /**
     * 打刻リクエストを受け取ってDBに保存し、同時にカレンダーテーブルにも同期する
     */
    public void savePunch(String accountId, String type) {
        // 1. 【既存処理】ログ用の打刻インスタンスを生成して保存
        AttendanceRecord record = new AttendanceRecord();
        record.setAccountId(accountId);
    }
}
```

```

String punchType = (type != null) ? type.trim() : "";
LocalDateTime now = LocalDateTime.now(); // 時刻を固定

if ("CLOCK_IN".equalsIgnoreCase(punchType) || "clock-in".equalsIgnoreCase(punchType)) {
    record.setType("clock-in");
    record.setClockIn(now);
    record.setStatus("CLOCKED_IN");
} else if ("CLOCK_OUT".equalsIgnoreCase(punchType) || "clock-out".equalsIgnoreCase(punchType)) {
    record.setType("clock-out");
    record.setClockOut(now);
    record.setStatus("CLOCK_OUT");
} else {
    throw new IllegalArgumentException("不正な打刻タイプです: " + type);
}

repository.save(record); // ログテーブルへの保存完了
// =====
// 2. 【新規追加】カレンダーテーブル (EntryExitCalendar) へのリアルタイム同期トラフィック
// =====
LocalDate today = LocalDate.now(); // 今日の日付 (yyyy-MM-dd)
String timeStr = now.format(DateTimeFormatter.ofPattern("HH:mm:ss")); // 打刻した時刻 (hh:mm:ss)

// まず、今日のカレンダー行がすでに存在するかチェックします
Optional<EntryExitCalendar> calendarOpt = calendarRepository.findByRegisteredAccountIdAndRecordDate(accountId, today);

EntryExitCalendar calendarRow;

```

```
if (calendarOpt.isPresent()) {
    // すでに今日中に打刻などのデータが存在していれば、その行を上書き対象にする
    calendarRow = calendarOpt.get();
} else {
    // まだ今日の行が作られていなければ、新しく1行を立ち上げる
    calendarRow = new EntryExitCalendar();

    // 仕様書ルールの record_id を生成
    String timePart = now.format(DateTimeFormatter.ofPattern("yyyyMMddHHmmssSSS"));
    String uuidPart = UUID.randomUUID().toString().substring(32);
    calendarRow.setRecordId("REC" + timePart + uuidPart);

    calendarRow.setRecordDate(today);
    calendarRow.setRegisteredAccountId(accountId);
    calendarRow.setEntryTime("-"); // 初期状態はハイフン
    calendarRow.setExitTime("-");
}

// 打刻のタイプ（出勤か退勤か）に応じて、確定時刻（Entry / Exit）の該当する方をパチッと更新します
if ("clock-in".equals(record.getType())) {
    calendarRow.setEntryTime(timeStr); // 確定Entry時刻を上書き
} else if ("clock-out".equals(record.getType())) {
    calendarRow.setExitTime(timeStr); // 確定Exit時刻を上書き
}
```

```
// カレンダーテーブルの翻訳窓口を通じてデータベースへ保存！
calendarRepository.save(calendarRow);
System.out.println("=== [Traffic Sync] カレンダーへのリアルタイム打刻同期が完了しました (" + record.getType() + ") ===");
}

/* --- 以下の getAttendanceStatus 等のメソッドはそのまま残す --- */
public String getAttendanceStatus(String accountId) {
    Optional<AttendanceRecord> lastRecord = repository.findFirstByAccountIdOrderByCreatedAtDesc(accountId);
    if (lastRecord.isEmpty()) return "CLOCKED_OUT";
    if ("clock-in".equals(lastRecord.get().getType())) return "CLOCKED_IN";
    return "CLOCKED_OUT";
}
}
```

同期トラフィックの動作確認テスト

コードを保存して、SpringBootを一度再起動してみてください。

再起動が完了したら、以下の手順でトラフィックが繋がったことを実感していただけます。

- 打刻を実行する

- 打刻画面（AttendanceBoard.vue）で「出勤」または「退勤」ボタンを押します。

- **裏側の連動**

- コンソールログに `=== [Traffic Sync] カレンダーへのリアルタイム打刻同期が完了しました ===` とパッと表示されます。

- **編集申請画面へ遷移する**

- そのままダッシュボードから「打刻内容編集申請」画面を開いてみてください。

- **結果**

- 先ほど打刻したばかりのリアルタイムな時間が、当日の行の「確定 Entry」や「確定 Exit」の列にしっかりと埋め込まれて表示されるようになります！

2つの別々の機能（打刻と申請管理）が、裏側のデータベース構造を通じて1つの綺麗なエコシステム（生態系）として融合しました。